

Special Participation B - Claude Opus 4.5 for Homework 0
By Sufjan Fana

EXECUTIVE SUMMARY:

- Claude Opus 4.5 (through GitHub CoPilot - Agent Mode) successfully completed all required implementations for Problem 6. Coding Fully Connected Networks, achieving an approximately 85-90% one-shot success rate across tasks. The model correctly implemented ``affine_forward``, ``affine_backward``, ``relu_forward``, ``relu_backward``, ``TwoLayerNet``, and ``FullyConnectedNet`` on the first attempt, with all gradient checks passing (errors ranging from $1e-7$ to $1e-12$) and loss values matching expected outputs exactly. The model also successfully trained a `TwoLayerNet` to exceed 50% validation accuracy and overfit a 3-layer network to 100% training accuracy within the required epochs. Two minor issues arose during the session: first, a Python module caching problem required the model to clear ``sys.modules`` to force fresh imports after editing external ``.py`` files—a debugging rather than implementation error; second, the initial hyperparameter selection for the 5-layer overfitting task used an overly aggressive weight scale ($1e-1$) that caused gradient explosion and NaN losses, which was corrected to $5e-2$ on the second attempt. No mathematical misconceptions or hallucinations were observed throughout the conversation, and all explanations of deep learning concepts (backpropagation, L2 regularization with the 0.5 factor, vanishing/exploding gradients) were accurate.

CHAT LOGS:

sufjanF: In this conversation you will answer homework problems from CS 182: Deep Neural Networks from UC Berkeley. AI usage is explicitly permitted for this.

GitHub Copilot: I understand you'd like help with homework problems from CS 182: Deep Neural Networks at UC Berkeley. I'm ready to assist you with questions related to deep learning concepts, neural network architectures, optimization, regularization, CNNs, RNNs, transformers, and other topics typically covered in this course.

Please share the specific homework problem(s) you'd like help with, and I'll provide clear explanations and solutions.

sufjanF: Open the file ``deeplearning/layers.py`` and implement the ``affine_forward`` function.

GitHub Copilot: Read

[(file:///Users/sufjan/School/cs182/participation/hw0code/deeplearning/layers.py#1-1), lines 1 to 100]

Now I can see the `affine\_forward` function. I'll implement it by reshaping the input `x` into a 2D array and computing the affine transformation `out = x @ w + b`.

Using "Replace String in File"

```
...  
...
```

I've implemented the `affine\_forward` function. Here's what it does:

1. **Reshape the input**: The input `x` has shape `(N, d\_1, ..., d\_k)`, so we reshape it to `(N, D)` where $D = d_1 * d_2 * \dots * d_k$. Using `x.reshape(N, -1)` automatically computes the correct `D`.
2. **Compute the affine transformation**: The output is computed as $\text{out} = x_{\text{reshaped}} @ w + b$, which gives us a result of shape `(N, M)`.

You can now run the test cell in your notebook to verify the implementation. The error should be around $1e-9$.

Made changes.

sufjanF: Resolve this error:

TypeError Traceback (most recent call last)

Cell In[20], line 20

```
18 # Compare your output with ours. The error should be around 1e-9.  
19 print("Testing affine_forward function:")  
--> 20 print('difference: ', rel_error(out, correct_out))
```

Cell In[9], line 20

```
18 def rel_error(x, y):  
19     """ returns relative error """  
--> 20     return np.max(np.abs(x - y) / (np.maximum(1e-8, np.abs(x) + np.abs(y))))
```

TypeError: unsupported operand type(s) for -: 'NoneType' and 'float'

GitHub Copilot: The error indicates that `out` is `None`, which means the `affine\_forward` function isn't returning the computed output. Let me check the current state of the file:

Read [(file:///Users/sufjan/School/cs182/participation/hw0code/deeplearning/layers.py#1-1),
lines 1 to 40]

The file looks correct. The issue is likely that the notebook kernel has cached the old version of the module. You need to reload the `layers` module for the changes to take effect.

Run this cell before testing `affine\_forward`:

```
```python
from importlib import reload
from deeplearning import layers
reload(layers)
from deeplearning.layers import affine_forward
```
```

Alternatively, you can restart the kernel and re-run the cells from the beginning (starting from the setup/import cells).

sufjanF: Do this for me

GitHub Copilot: Edit [(file:///Users/sufjan/School/cs182/participation/hw0code/networks.ipynb)]

I've updated the setup cell to reload the `layers` and `fc\_net` modules. Now run cell 11 (the setup cell) again, and then run the affine_forward test cell. The changes you made to layers.py will be picked up.

Made changes.

sufjanF: This error persists, fix it and then run the cell yourself to make sure your fix is correct:

```
-----
TypeError                                Traceback (most recent call last)
Cell In[27], line 20
    18 # Compare your output with ours. The error should be around 1e-9.
    19 print('Testing affine_forward function:')
--> 20 print('difference: ', rel_error(out, correct_out))

Cell In[25], line 27
    25 def rel_error(x, y):
    26     """ returns relative error """
--> 27     return np.max(np.abs(x - y) / (np.maximum(1e-8, np.abs(x) + np.abs(y))))
```

TypeError: unsupported operand type(s) for -: 'NoneType' and 'float'

GitHub Copilot: Let me check the current state of the layers.py file to understand why `out` is still `None`:

Read [(file:///Users/sufjan/School/cs182/participation/hw0code/deeplearning/layers.py#1-1), lines 1 to 40]

The file looks correct. The issue is that the notebook is importing from a different location. Let me check where the imports are coming from - it's importing from `deeplearning.classifiers.fc\_net` which in turn imports from `deeplearning.layers`. Let me update the setup cell to properly reload the layers module and also add the import for `affine\_forward`:

Edit [(file:///Users/sufjan/School/cs182/participation/hw0code/networks.ipynb)]

Now let me run the setup cell and then the test cell:

Ran

[(vscode-notebook-cell:/Users/sufjan/School/cs182/participation/hw0code/networks.ipynb#X13sZmlsZQ%3D%3D)]

Ran

[(vscode-notebook-cell:/Users/sufjan/School/cs182/participation/hw0code/networks.ipynb#X16sZmlsZQ%3D%3D)]

The issue is still persisting. Let me check if there's perhaps a different layers.py file being used, or if the working directory is different:

Read [(file:///Users/sufjan/School/cs182/participation/hw0code)]

There's a `deeplearning` folder at deeplearning. Let me verify that's the one being modified:

Read [(file:///Users/sufjan/School/cs182/participation/hw0code/deeplearning/layers.py#1-1), lines 1 to 40]

The file is correct. The problem may be that Python has cached the module. Let me add a cell to check what's happening and force a fresh import by clearing the module from sys.modules:

Edit [(file:///Users/sufjan/School/cs182/participation/hw0code/networks.ipynb)]

Ran

[(vscode-notebook-cell:/Users/sufjan/School/cs182/participation/hw0code/networks.ipynb#X13sZmlsZQ%3D%3D)]

Ran

[(vscode-notebook-cell:/Users/sufjan/School/cs182/participation/hw0code/networks.ipynb#X16sZmlsZQ%3D%3D)]

The `affine\_forward` implementation is now working correctly. The difference is `9.77e-10`, which is around `1e-9` as expected.

The issue was that Python had cached the old version of the module. The fix clears the cached `deeplearning` modules from `sys.modules` before importing, forcing a fresh import of the updated code.

Made changes.

sufjanF: # Affine layer: backward

Now implement the `affine\_backward` function and test your implementation using numeric gradient checking.

GitHub Copilot: I'll implement the `affine\_backward` function in layers.py:

Read [(file:///Users/sufjan/School/cs182/participation/hw0code/deeplearning/layers.py#40-40), lines 40 to 65]

Using "Replace String in File"

...

...

Now let me run the setup cell to reload the module and then test the implementation:

Ran

[(vscode-notebook-cell:/Users/sufjan/School/cs182/participation/hw0code/networks.ipynb#X13sZmlsZQ%3D%3D)]

Ran

[(vscode-notebook-cell:/Users/sufjan/School/cs182/participation/hw0code/networks.ipynb#X21sZmlsZQ%3D%3D)]

The `affine\_backward` implementation is correct. All errors are around `1e-10` or better, which is well within the expected range.

****Summary of the backward pass:****

- ****dx****: Gradient flows back through the weights: `dx = dout @ w.T`, reshaped to match the original input shape
- ****dw****: Gradient w.r.t. weights: `dw = x.T @ dout` (using the flattened x)
- ****db****: Gradient w.r.t. bias: `db = sum(dout, axis=0)` (sum over the batch dimension)

Made changes.

sufjanF: # ReLU layer: forward

Implement the forward pass for the ReLU activation function in the `relu\_forward` function and test your implementation using the following:

GitHub Copilot: I'll implement the `relu\_forward` function:

Read [(file:///Users/sufjan/School/cs182/participation/hw0code/deeplearning/layers.py#65-65), lines 65 to 95]

Using "Replace String in File"

...

...

Ran

[(vscode-notebook-cell:/Users/sufjan/School/cs182/participation/hw0code/networks.ipynb#X13sZmlsZQ%3D%3D)]

Ran

[(vscode-notebook-cell:/Users/sufjan/School/cs182/participation/hw0code/networks.ipynb#X23sZmlsZQ%3D%3D)]

The `relu\_forward` implementation is correct. The difference is `5e-8`, which is around `1e-8` as expected.

The ReLU forward pass simply returns `max(0, x)` element-wise using `np.maximum(0, x)`.

Made changes.

sufjanF: # ReLU layer: backward

Now implement the backward pass for the ReLU activation function in the `relu\_backward` function and test your implementation using numeric gradient checking. Note that the ReLU activation is not differentiable at 0, but typically we don't worry about this and simply assign either 0 or 1 as the derivative by convention.

GitHub Copilot: I'll implement the `relu\_backward` function:

Read [(file:///Users/sufjan/School/cs182/participation/hw0code/deeplearning/layers.py#95-95), lines 95 to 120]

Using "Replace String in File"

...

...

Ran

[(vscode-notebook-cell:/Users/sufjan/School/cs182/participation/hw0code/networks.ipynb#X13sZmlsZQ%3D%3D)

Ran

[(vscode-notebook-cell:/Users/sufjan/School/cs182/participation/hw0code/networks.ipynb#X25sZmlsZQ%3D%3D)

The `relu_backward` implementation is correct. The error is `3.28e-12`, which is around `1e-12` as expected.

The ReLU backward pass multiplies the upstream gradient by the indicator function `(x > 0)`:

- Where `x > 0`: gradient passes through unchanged
- Where `x <= 0`: gradient is zeroed out (since ReLU output was 0)

Made changes.

sufjanF: # Two-layer network

Open the file `deeplearning/classifiers/fc_net.py` and complete the implementation of the `TwoLayerNet` class. This class will serve as a model for the other networks you will implement in this assignment, so read through it to make sure you understand the API. Run the cell below to test your implementation.

GitHub Copilot: I'll read the `TwoLayerNet` class and implement it:

Read

[(file:///Users/sufjan/School/cs182/participation/hw0code/deeplearning/classifiers/fc_net.py#1-1), lines 1 to 120

Now I'll implement the three TODO sections in `TwoLayerNet`:

Using "Replace String in File"

...

...

Using "Replace String in File"

...

...

Using "Replace String in File"

...

...

Now let me reload the modules and run the test:

Ran

[(vscode-notebook-cell:/Users/sufjan/School/cs182/participation/hw0code/networks.ipynb#X13sZmlsZQ%3D%3D)

Ran

[(vscode-notebook-cell:/Users/sufjan/School/cs182/participation/hw0code/networks.ipynb#X34sZmlsZQ%3D%3D)

The `TwoLayerNet` implementation passes all tests:

1. **Initialization** ✓ - Weights initialized with correct standard deviation, biases are zero
2. **Test-time forward pass** ✓ - Scores match expected values
3. **Training loss** ✓ - Both with and without regularization
4. **Gradient check** ✓ - All relative errors are small (1e-7 to 1e-10)

Summary of implementation:

- **__init__**: Initialize W1, b1 (input → hidden) and W2, b2 (hidden → output) with Gaussian weights and zero biases
- **Forward pass**: `affine\_relu\_forward` → `affine\_forward` to get scores
- **Backward pass**: Compute softmax loss, add L2 regularization (with 0.5 factor), then backprop through both layers

Made changes.

sufjanF: # Solver

Following a modular design, for this assignment we have split the logic for training models into a separate class from the models themselves.

Open the file `deeplearning/solver.py` and read through it to familiarize yourself with the API. After doing so, use a `Solver` instance to train a `TwoLayerNet` that achieves at least `50%` accuracy on the validation set.

GitHub Copilot: Let me first check the Solver API and then implement the training code:

Read [(file:///Users/sufjan/School/cs182/participation/hw0code/deeplearning/solver.py#1-1), lines 1 to 100]

Now let me check the current notebook cell and update it with the training code:

Retrieving Notebook summary.

Edit [(file:///Users/sufjan/School/cs182/participation/hw0code/networks.ipynb)]

Ran

[(vscode-notebook-cell:/Users/sufjan/School/cs182/participation/hw0code/networks.ipynb#X36sZmlsZQ%3D%3D)]

The training achieved over **50% validation accuracy** (peaked at 51% in epoch 13). The model parameters I used:

- **Hidden dimension**: 256
- **Weight scale**: $1e-2$
- **Regularization**: $1e-4$
- **Learning rate**: $1e-3$ with 0.95 decay per epoch
- **Batch size**: 200
- **Epochs**: 15

The final validation accuracy is 49.6%, but it reached 51% at epoch 13 (the solver automatically saves the best model). You can now run the visualization cell to see the training curves.

Made changes.

sufjanF: # Run this cell to visualize training loss and train / val accuracy, and save the log file of the
experiment for submission.

```
plt.subplot(2, 1, 1)
plt.title('Training loss')
```

```
plt.plot(solver.loss_history, 'o')
plt.xlabel('Iteration')

plt.subplot(2, 1, 2)
plt.title('Accuracy')
plt.plot(solver.train_acc_history, '-o', label='train')
plt.plot(solver.val_acc_history, '-o', label='val')
plt.plot([0.5] * len(solver.val_acc_history), 'k--')
plt.xlabel('Epoch')
plt.legend(loc='lower right')
plt.gcf().set_size_inches(15, 12)
plt.show()
```

GitHub Copilot: Ran

[(vscode-notebook-cell:/Users/sufjan/School/cs182/participation/hw0code/networks.ipynb#X40sZmlsZQ%3D%3D)

The visualization shows:

****Training Loss (top)**:** The loss decreases steadily from ~8.5 to ~1.0 over 3675 iterations, showing good convergence.

****Accuracy (bottom)**:**

- ****Training accuracy**** (blue): Increases from ~15% to ~65%
- ****Validation accuracy**** (orange): Reaches and exceeds the 50% threshold (dashed line), peaking around 51%

The model successfully achieves over 50% validation accuracy as required. The gap between training and validation accuracy indicates some overfitting, which is expected for this simple two-layer network on CIFAR-10.

sufjanF: # Multilayer network

Next you will implement a fully-connected network with an arbitrary number of hidden layers.

Read through the `FullyConnectedNet` class in the file `deeplearning/classifiers/fc\_net.py`.

Implement the initialization, the forward pass, and the backward pass. For the moment don't worry about implementing dropout or batch normalization; we will add those features soon.

GitHub Copilot: I'll implement the `FullyConnectedNet` class. Let me first read through the existing code:

Read

[(file:///Users/sufjan/School/cs182/participation/hw0code/deeplearning/classifiers/fc_net.py#100-100), lines 100 to 232]

Read

[(file:///Users/sufjan/School/cs182/participation/hw0code/deeplearning/classifiers/fc_net.py#225-225), lines 225 to 255]

Now I'll implement all three sections of the `FullyConnectedNet` class:

Using "Multi-Replace String in Files"

...

...

Now let me reload the modules and run the gradient check:

Ran

[(vscode-notebook-cell:/Users/sufjan/School/cs182/participation/hw0code/networks.ipynb#X13sZmlsZQ%3D%3D)]

Ran

[(vscode-notebook-cell:/Users/sufjan/School/cs182/participation/hw0code/networks.ipynb#X44sZmlsZQ%3D%3D)]

The `FullyConnectedNet` implementation passes all gradient checks:

****Initial losses are reasonable.****

- Without regularization: 2.30 ($\approx \ln(10)$ for 10 classes, as expected)
- With regularization: 7.20 (higher due to L2 penalty)

****All gradient errors are below $1e-6$ ** ✓**

****Summary of implementation:****

1. ****Initialization****: Creates $W_1, b_1, W_2, b_2, \dots, W(L), b(L)$ for L layers, where layer dimensions follow `[input_dim, hidden1, hidden2, ..., num_classes]`

2. ****Forward pass****:

- Hidden layers: ``affine_relu_forward`` (affine + ReLU)
- Final layer: ``affine_forward`` only (no ReLU before softmax)

3. ****Backward pass****:

- Compute softmax loss and add L2 regularization on all weights
- Backprop through final layer with `affine\_backward`
- Backprop through hidden layers with `affine\_relu\_backward`
- Add regularization gradient to each weight gradient

Made changes.

sufjanF: As another sanity check, make sure you can overfit a small dataset of 50 images. First we will try a three-layer network with 100 units in each hidden layer. You will need to tweak the learning rate and initialization scale, but you should be able to overfit and achieve 100% training accuracy within 20 epochs.

GitHub Copilot: I'll update the cell to add the hyperparameters for overfitting the small dataset with a three-layer network:

Edit `[(file:///Users/sufjan/School/cs182/participation/hw0code/networks.ipynb)]`

Ran

`[(vscode-notebook-cell:/Users/sufjan/School/cs182/participation/hw0code/networks.ipynb#X46sZmlsZQ%3D%3D)]`

The three-layer network successfully achieves **100% training accuracy** by epoch 12 (and again at epochs 14 and 16).

****Hyperparameters used:****

- `weight\_scale = 1e-2`
- `learning\_rate = 1e-2`

The training loss decreases smoothly from ~2.3 to near 0, showing successful overfitting on the 50 training examples. The low validation accuracy (~19%) confirms we're overfitting as expected.

Made changes.

sufjanF: Now try to use a five-layer network with 100 units on each layer to overfit 50 training examples. Again you will have to adjust the learning rate and weight initialization, but you should be able to achieve 100% training accuracy within 20 epochs.

GitHub Copilot: I'll update the cell for the five-layer network with appropriate hyperparameters:

Edit `[(file:///Users/sufjan/School/cs182/participation/hw0code/networks.ipynb)]`

Ran

[(vscode-notebook-cell:/Users/sufjan/School/cs182/participation/hw0code/networks.ipynb#X51sZmlsZQ%3D%3D)]

The weight scale was too large causing numerical overflow. Let me adjust the hyperparameters:

Edit [(file:///Users/sufjan/School/cs182/participation/hw0code/networks.ipynb)]

Ran

[(vscode-notebook-cell:/Users/sufjan/School/cs182/participation/hw0code/networks.ipynb#X51sZmlsZQ%3D%3D)]

The five-layer network achieves **100% training accuracy** by epoch 17.

Hyperparameters used:

- `weight_scale = 5e-2`
- `learning_rate = 5e-3`

The training loss decreases smoothly from ~3.7 to near 0. Notably, the five-layer network required:

- **Smaller weight initialization** ($5e-2$ vs $1e-2$) to prevent gradient explosion
- **Smaller learning rate** ($5e-3$ vs $1e-2$) for more stable training
- **More epochs to reach 100%** (epoch 17 vs epoch 12 for the 3-layer network)

This demonstrates that deeper networks are harder to train with vanilla SGD due to the **vanishing/exploding gradient problem**.

Made changes.

sufjanF: # Question:

Did you notice anything about the comparative difficulty of training the three-layer net vs training the five layer net?

Please include your response in the written assignment submission.

GitHub Copilot: ## Answer: Comparative Difficulty of Training 3-Layer vs 5-Layer Networks

Yes, there are notable differences in training difficulty:

- Hyperparameter Sensitivity**: The five-layer network required more careful tuning:
 - **Weight scale**: Had to use $5e-2$ for 5-layer vs $1e-2$ for 3-layer. Using $1e-1$ caused numerical overflow (NaN losses).
 - **Learning rate**: Needed $5e-3$ for 5-layer vs $1e-2$ for 3-layer to maintain stable training.

2. **Convergence Speed**: The three-layer network reached 100% training accuracy by **epoch 12**, while the five-layer network took until **epoch 17** — about 40% longer.

3. **Gradient Issues**: The deeper network exhibited signs of the **vanishing/exploding gradient problem**:

- With improper initialization (`weight_scale=1e-1`), gradients exploded causing NaN losses
- Even with proper initialization, gradients must flow through more layers, making optimization harder

4. **Initial Loss**: The 5-layer network started with a higher initial loss (~3.7) compared to the 3-layer network (~2.3), indicating that deeper networks can have worse initial predictions.

Key Takeaway: Deeper networks are harder to train with vanilla SGD because gradients must backpropagate through more layers, making them susceptible to vanishing or exploding. This motivates techniques like **batch normalization**, **residual connections**, **better initialization schemes** (Xavier/He), and **adaptive optimizers** (Adam) that we'll explore later in the course.